Part III

Linux networking

Secure connections - ssh

- Assuming you have a Linux/Unix-like terminal (WSL/Mac OS X), then:
- To log in to another computer over the Internet, use:
  `$ ssh pmb39@apollo.lsc.phy.private.cam.ac.uk`
  `pmb39@apollo's password:` [*Enter password.*]
  `pmb39@apollo $`
  and you will get a terminal that is running on the remote machine (assuming you have appropriate permissions).
- Note your username may not be the same on the remote machine as on your local machine (hence specifying `pmb39`).
- Here you can run any programs that are installed on that computer.
- Any files you create will be on that computer only (unless any directories are somehow shared with others).

Password changing - passwd

- When logging into a new system, always change your initial password. It might have been stored by a lazy/nefarious sys-admin.
- To change your password:

```
$ passwd
Enter login(LDAP) password:
New password:
Re-enter new password:
```

- Make sure you use a strong password ($\geqslant 16$ characters, upper-case, lower-case, numbers, punctuation). The sys-admin may enforce some constraints automatically.
- Memorise the password or store it in your password-manager.
- Once you have changed your password, you can logout of the remote computer with `exit`.

ssh fingerprints

The very first time you ssh to a new system, you should see something like: `ssh cerberus1`

```
The authenticity of host 'cerberus1 (172.24.62.68)' can't
be established.

RSA key fingerprint is
SHA256:s/soi65/q8DiXZDQee+opZ0Zmu2Rta7RNTpys2iCWLc.
This key is not known by any other names
Are you sure you want to continue connecting
(yes/no/[fingerprint])?
```

You should type `yes` at this point.
(or not, depending on your level of trust)

ssh fingerprints

- On the first ssh to a new machine, `ssh` checks the public encryption key and prints and stores its fingerprint.
- On connecting to the same machine later, `ssh` will check the fingerprint.
- If it differs, then someone has intercepted the connection, or redirected it to a different machine, and the client will warn you.
- Unless you have another way of checking you're connecting to the correct machine when you first connect, enter "yes" at the prompt.
- For some systems, the RSA fingerprints are published online, so that you can check the website and compare the keys.
- If they differ, you know something is wrong.
- If they match then either everything is fine, or someone has infiltrated both the system and the website.
- See `https://www-internal.lsc.phy.cam.ac.uk/systems.shtml`

Running multiple remote terminals - screen/tmux

- What happens if your ssh-connection goes down (wireless disconnects, you close your laptop)?
- Your remote shell will end and you can't reconnect to what you were doing.
- The answer is a multiplexer such as `screen`, `tmux`, or `byobu`.
- You can open multiple terminals on a remote machine (e.g. one to compile, one to edit source-code, one to run simulations)
- This turns one remote terminal (over one ssh connection) into multiple that you can switch between with keyboard shortcuts.
- Helpfully, the multiplexer continues running even if the `ssh` connection fails, and you can reconnect to the session later.
- I use this to keep work open between working in the office versus working from home.

Running multiple remote terminals

To start a multiplexing session: `screen`

Then `screen` commands:

- Ctrl-a Ctrl-c : Create a new terminal.
- Ctrl-a 0 : Switch to terminal 0 (or 1-9).
- Ctrl-a Ctrl-d : Detach from the screen session
- Ctrl-a k : Kill current terminal
- Ctrl-a [: Enter scroll-mode; ↑ / ↓ to scroll; q to quit scroll-mode.
- Ctrl-a ? : Screen Help

The shortcuts for `tmux` are more-or-less the same, except with Ctrl-b instead of Ctrl-a.

Running multiple remote terminals ctd

To reconnect to an existing screen when you've logged back in:

```
$ screen -list

There are screens on:

1552000.Lecture_Slides (24/08/23 08:31:17) (Attached)
57458.Assignments_Lectures (11/08/23 15:42:13) (Attached)
54334.AMReX-work (11/08/23 15:40:28) (Attached)

3 Sockets in /run/screen/S-pmb39.

$ screen -x -r 1552000
```

Or omit the screen ID if there's only one running.

X-forwarding

- If you try to run a graphical/non-text program via ssh:
```
pmb39@h2g2 $ ssh pmb39@apollo
pmb39@apollo $ gedit
Cannot open display:
pmb39@apollo $
```
- By default `ssh` only opens a text-terminal. It does not forward GUI information from the remote machine. Instead, use:
`ssh -X pmb39@apollo`
- However, the GUI of remotely running programs will run slowly as all the pixel-information must be transferred to your screen across the Internet.
- Remote Desktops are a more efficient alternative, but outside the scope of this lecture.

Not X-forwarding

- Avoid needing to use `ssh -X` by learning how to use command-line based programs such as `emacs/vim` or ones that run in client-server mode such as VSCode.
- `ssh -X` may need more work if you're using Windows or Mac.
- For Windows: MobaXTerm `https://mobaxterm.mobatek.net/`
- I recommend making sure you can connect without using VSCode; if you run out of disk-space, the remote server is overloaded, or there's a weird error, it's easier to debug from pure `ssh`.

Secure copy - scp

- The `cp` command only works for files directly accessible on the same computer, i.e. anywhere you can `cd` to in one terminal.
- To copy files from/to remote machines, use:

```
pmblakely@h2g2 $ scp ./settingsFile pmb39@apollo:~/

pmblakely@h2g2 $ scp -r -C pmb39@apollo:~/outputData ./
```

  to copy a settings file across, and then recursively (`-r`) with compression (`-C`) copy the output directory back.
- One of the computers involved in the copy must be the one you are currently logged into (or you need the `-3` option).
- It is usually better to run `scp` on your local machine, but make sure you get the source (first) and destination (second) files the right way round.
- It is unlikely that you can ssh from `apollo` to your laptop, so `scp` must be run on your laptop, for either transfer direction.

Synchronizing folders - rsync

- If your connection is liable to drop out while copying, or you need to incrementally re-synchronize a particular folder between two machines, use `rsync`.
- This uses heuristics (file-size, modification-time, etc.) to see if files have changed since last run and only transfers modified files.
- `rsync -a ./MyCode/ pmb39@apollo:~/MyCode/`

  will update files in `MyCode` on `apollo` to match those on my local machine.
- `-a` means "archive" mode, which is probably what you want for synchronizing directories. See `man rsync` for many more options.
- Make sure your folder-names end with `/` If not, `rsync` will probably not do what you want (e.g. create a new folder inside the one you want to transfer to).
- A better way to synchronize source-code across computers is to

ssh keys

- You can set up ssh-keys to make logging into machines more secure.

```
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key
(/home/pmblakely/.ssh/id_rsa):
Enter passphrase:  My Secret Phrase
Enter same passphrase again:  My Secret Phrase
Your identification has been saved in
/home/pmblakely/.ssh/id_rsa
Your public key has been saved in
/home/pmblakely/.ssh/id_rsa.pub
```

- This stores a private/public key-pair (RSA algorithm) in two files.

ssh keys ctd

- You can now copy the (public) key from your local computer to a remote machine: `ssh-copy-id pmb39@apollo`
- When you attempt to `ssh` to that machine again, you will be logged in automatically as the remote machine checks that you have the private key corresponding to the public key on that machine.
- If the private key (on your local machine) is compromised (e.g. your laptop is stolen), the attacker then has access to the remote machine. This is why we used a passphrase to encrypt the ssh-key.
- The commands `scp` and `rsync` use `ssh` to make the connection and transfer data.
- Other programs may also use `ssh` to connect to remote machines, e.g. VSCode, VisIt, etc.

Compression - tar

- If transferring large amounts of data, even a 1GiB connection may not be fast enough, so you need to compress it.
- You can use `tar` (short for `tape-archive`) as:

  ```
  tar -cjf MyCode.tar.bz2 ./AllMyCode ./AllMySettings
  ```

  to create (`-c`) a bzipped (`-j`) tar-file called (`-f`) MyCode.tar.bz2 from the folders `AllMyCode` and `AllMySettings`.
- Alternative compression formats include `gzip`:
  ```
  tar -czf MyCode.tar.gz ./AllMyCode
  ```
- (bzip is just a compression algorithm; it usually produces a smaller file)

De-compression

- To decompress (eXtract) files:
  `tar -xf MultiPhysics_Source_Code.tar.gz`
- To list what is in the file first:
  `tar -tf MultiPhysics_Source_Code.tar.gz`
- You can pass `-j` or `-z` explicitly, but `tar` can probably work it out by itself.
- Alternatively `zip` and `unzip` handle `.zip` files (originally from PKWARE, most common on Windows):
  `zip -r MyCode.zip ./MyCode/`
  `unzip MyCode.zip`

Stopping and pausing programs

- If you want to kill a program running in a terminal, press Ctrl-C.
- This will not work if the programmer has disabled it (fairly unlikely).
- To pause a running program, press Ctrl-Z.
- To allow a paused to continue to run in the background, while you continue to use the terminal, type `bg`.
- If you need to put a program in the foreground again, type `fg`.
- To run a program and immediately background it, run it as:
  `xclock &`
- To ensure a program does not stop when your ssh-connection ends:
  `nohup myLongCode &`
  which stops the program from responding to the Hang-UP (HUP) signal.

Machine characteristics

- If the sys-admin has not documented their network properly, you can still find out what hardware machines have.
- `lscpu` or `cat /proc/cpuinfo` identifies the CPU(s) in the computer.
- `free -m` gives the free memory (RAM) in units of MB.
- `df -h` gives the disk-space available on physical disks.
- Remember that RAM (Random Access Memory) is the temporary storage for currently running programs and their data. The disk (HDD or SSD) is permanent storage.
- Also try `lshw` for more hardware characteristics.
- If it has an NVIDIA GPU, `nvidia-smi` gives GPU-usage details.

CSC Network Specifics

Different Linux networks have their own ways of organising things, and the following are specific to the CSC Network.

More information can be found at `www-internal.lsc.phy.cam.ac.uk`

Please read this before coming to ask for help.

sudo

- Many online examples suggest solutions starting with:
  `sudo apt-get install shellcheck`
  or similar.
- If we try this, we get:

```
[sudo] password for pmb39: ********
Sorry, user pmb39 is not allowed to execute
'/usr/bin/apt-get install shellcheck' as root
on apollo.
```

- `sudo` is short for: super-user do
- You do not usually have super-user/root access to machines that are not your own.
- See `https://xkcd.com/149/`

Home directories

- Your home directory is of the form `/home/raid/pmb39`.
- It is shared across all CSC machines.
- You initially have 5GB of disk-space, and it is backed up daily.
- This should be used for code, settings files, your projects, and similar things that would cause major problems if lost.
- It should not be used for large amounts of data output that can easily be regenerated.
- Large amounts of data can be stored in the `/local/data/public` folders on all computers.
- These are mounted on all other machines as `/data/apollo` for example.

Home-directory disk-usage - quota/du

- Use `quota` to list your home-directory quota on the `maia` file-server. (You don't have ssh-access to `maia`.)

```
Disk quotas for user pmb39 (uid 8894):
Filesystem  blocks   quota   limit   grace files  quota limit
maia:/raid 23489480  25000000 26000000     280364      0     0
```

where the sizes are in KB.

- The `du` command calculates disk-usage for the current directory (and sub-directories):

```
$du -hc ./Group_Meetings/
 356K    Group_Meetings/Pictures_Jan_23
 1.6M    Group_Meetings/Pictures_220223
...
 122M    Group_Meetings/
 122M    total
```

where `-c` gives a final total, and `-h` makes sizes human-readable.

VSCode Remote SSH storage

- You will find that VS-Code puts a lot of temporary files in your home directory, filling it up needlessly.
- I (as sysadmin) have set environment variable `XDG_CACHE_HOME` so that any C++ temporary storage is put on the local disks.
- However, you will need to do the following:
  1. Open VSCode.
  2. Open Settings
  3. Search for "remote ssh server install"
  4. For the "Remote.SSH: Server Install Path" setting, add:

```
Item: MACHINE_NAME_HERE.lsc.phy.private.cam.ac.uk
Value: /local/data/public/YOUR_USERNAME/VSCode_Cache
```

Other disk-usage - df

- Use `df -h` to check overall disk usage:

```
$ df -h | grep -v "snap" | grep dev
Filesystem      Size  Used Avail Use% Mounted on
udev             32G     0   32G   0% /dev
/dev/nvme0n1p3   98G   39G   55G  42% /
tmpfs            32G  278M   32G   1% /dev/shm
/dev/nvme0n1p6  110G   29G   76G  28% /scratch
/dev/nvme0n1p4  9.8G  1.1G  8.3G  12% /tmp
/dev/nvme0n1p2  197M  6.1M  191M   4% /boot/efi
/dev/sda1       3.6T  2.9T  578G  84% /local/data
```

- where we ignore any disks mentioning "snap" because this is an Ubuntu-speciality that we want to ignore.
- On the CSC-Network, use the custom `quota-local` command to get per-user local disk-usage.

CSC computers

- `https://www-internal.lsc.phy.cam.ac.uk/systems.shtml` for a list of computers
- This includes details such as RAM, disk-space, Processor, etc.
- If you need to find a free computer, use `https://www-internal.lsc.phy.cam.ac.uk/mrtg` to see current processor usage.
- You can also use `top` and `htop` to see what processes are running on a machine.
- Or, use `ps aux` or `ps faux`
- To see who is logged in, use `who` or `last`.
- There is no job-queueing or enforced-limiting on these machines, so be careful you don't hog/overuse them.
- For long running jobs, you can reduce their priority by prefixing the command with `nice -19`.

CSC desktops

- As well as the servers, most of the desktops you see in the CSC areas of Maxwell are on the CSC network.
- You can log in to these with the same CSC password.
- Do not disconnect the desktops from the wired network, or switch them off.
- Someone may be running a simulation on them and be slightly annoyed...

VPNs

- You may have noticed that the full-name of `apollo` contains `private`. In Cambridge, this means it is not visible from outside the Cambridge network (CUDN).
- If you are outside the CUDN (e.g. in private accommodation or away from Cambridge), you need a VPN to reach CSC machines: `https://help.uis.cam.ac.uk/service/network-services/remote-access/uis-vpn`
- On a CSC laptop, run: `setup_vpn` to set up a VPN connection.
- If you get an error messages such as:

```
ssh: Could not resolve hostname athena.lsc.phy.private.cam.ac.uk
```

  then you need to use the machine's IP address instead:

```
ssh pmb39@172.24.62.48
```

  where the IP addresses are listed at:
  `https://www-internal.lsc.phy.cam.ac.uk/systems.shtml`

Laptops

- You connect to the Maxwell/West-Hub network using WiFi (eduroam or UniOfCam).
- If you are using a laptop, the data on it is not automatically backed-up. Either back it up yourself (e.g. to Google-Drive, Microsoft One Drive) perhaps using `rclone`, `rsync` to your CSC home directory.

Printing in CSC

- Present your University card to any Maxwell printer.
- Click “resend my invitation”.
- Type in your email address. Press Send.
- You will receive an email with a link to an .exe file (“Get started”), and a code to type into the printer
- Once installed, the printer will show as Physics PaperCut printer, and you will be able to print, copy and scan to your email address.
- At a Maxwell printer, present your University card, and you can release your print-jobs.
- Or, use `https://managedprint.uis.private.cam.ac.uk`
- From Linux you must use Google Chrome and the PaperCut-Hive extension (from the App Store).
- If logged in with your University-Google account, PaperCut will automatically allow printing to the Maxwell queue.

Installing new software

- If the software you need is not available already, you can:
  - Install it using `sudo` if you have permission.
  - Ask for it to be installed as an Ubuntu package `https://packages.ubuntu.com/`
  - Ask for it to be compiled from source.
  - Compile and install it yourself in `/local/data/public/pmb39/bin` for example.
- The same applies if you need a newer version than is available by default.
- Some extra software is in `/lsc/opt` already, such as newer versions of `gcc`, CUDA compilers, Intel compilers, and visualisation software VisIt.
- See `https://www-internal.lsc.phy.cam.ac.uk/software_noble.shtml`